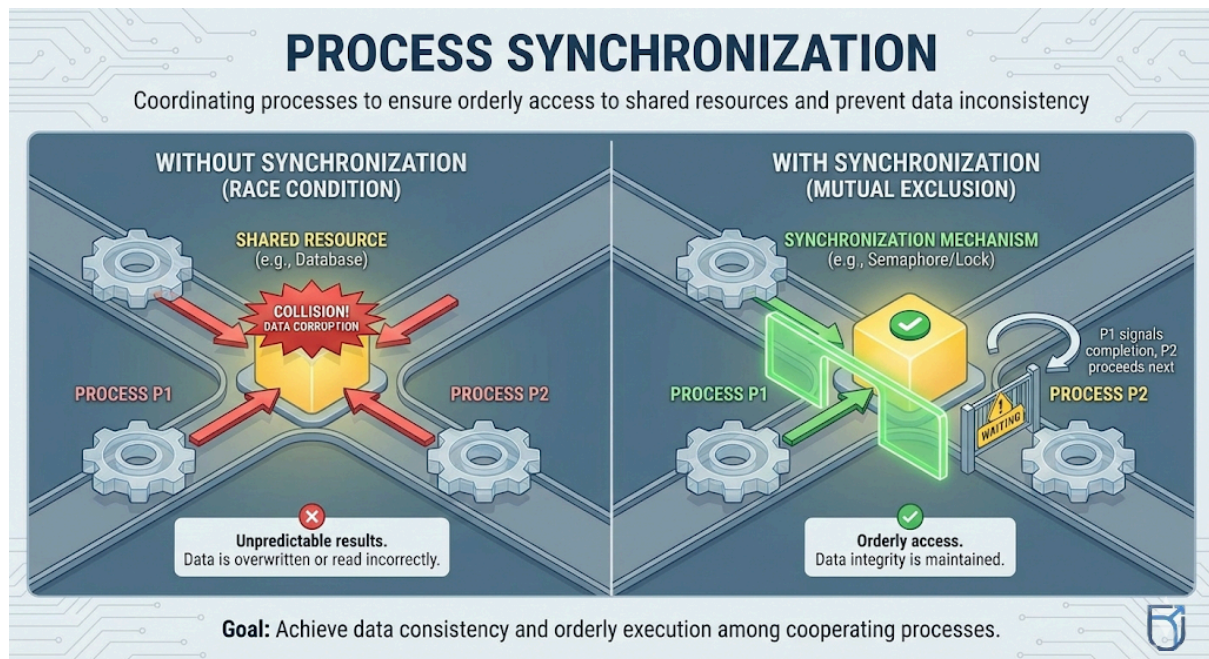


Lecture 10 NOTES – Process Synchronization I: Critical Section and Race Conditions

Introduction to Process Synchronization

Meaning:

Process synchronization is the act of coordinating multiple processes (or threads) that are running at the same time. It ensures they do not get in each other's way when accessing shared resources (like variables, files, or memory).



- **Simple Analogy:** Think of a single-lane bridge. Synchronization is the traffic light that ensures cars from opposite directions don't crash on the bridge.

Need for Synchronization:

- **Data Consistency:** prevents data from becoming corrupted or nonsensical.
- **Orderly Execution:** ensures processes run in the correct sequence when necessary (e.g., Process B shouldn't print a file until Process A has finished generating it).

The Problem of Shared Resources (The "Critical Section"):

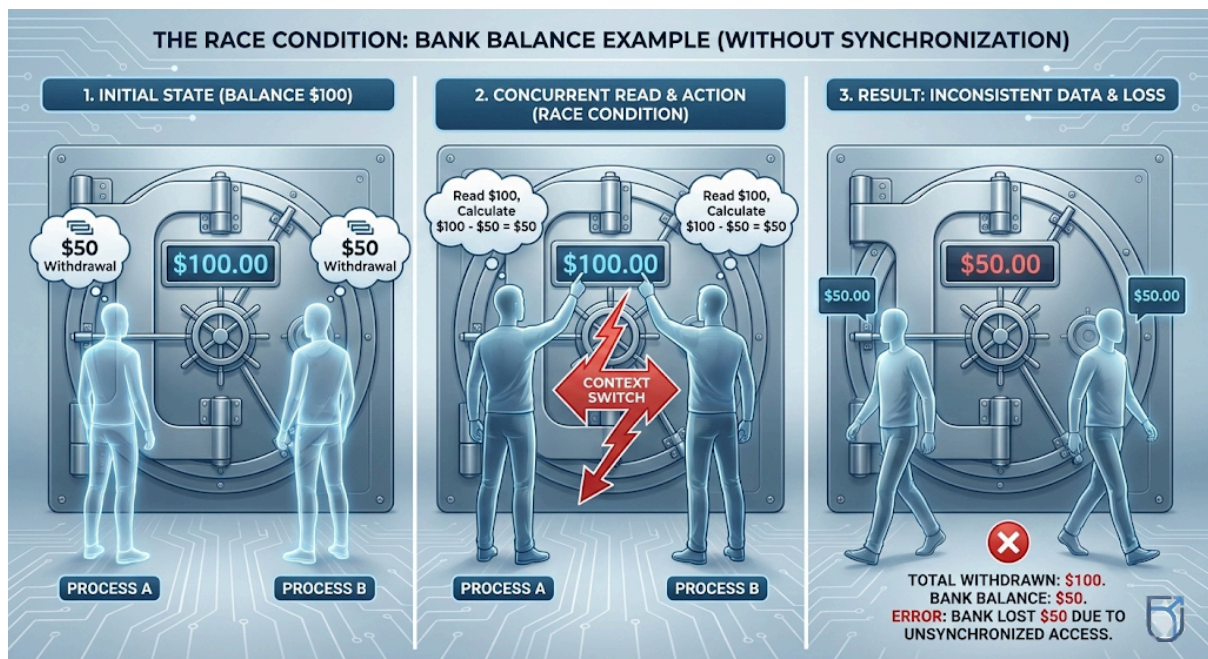
- If two processes try to write to the same shared variable at the exact same time, the final value depends on who "wins" the race.
- This leads to **non-deterministic outcomes** (the result changes every time you run the program).

Real-World Motivation: The Bank Balance Example

This is the classic "Race Condition" problem used to explain why we need synchronization.

Scenario:

- **Shared Resource:** A bank account with a balance of **\$100**.
- **Process A:** Wants to withdraw \$50.
- **Process B:** Wants to withdraw \$50.



What happens WITHOUT Synchronization (The Race Condition):

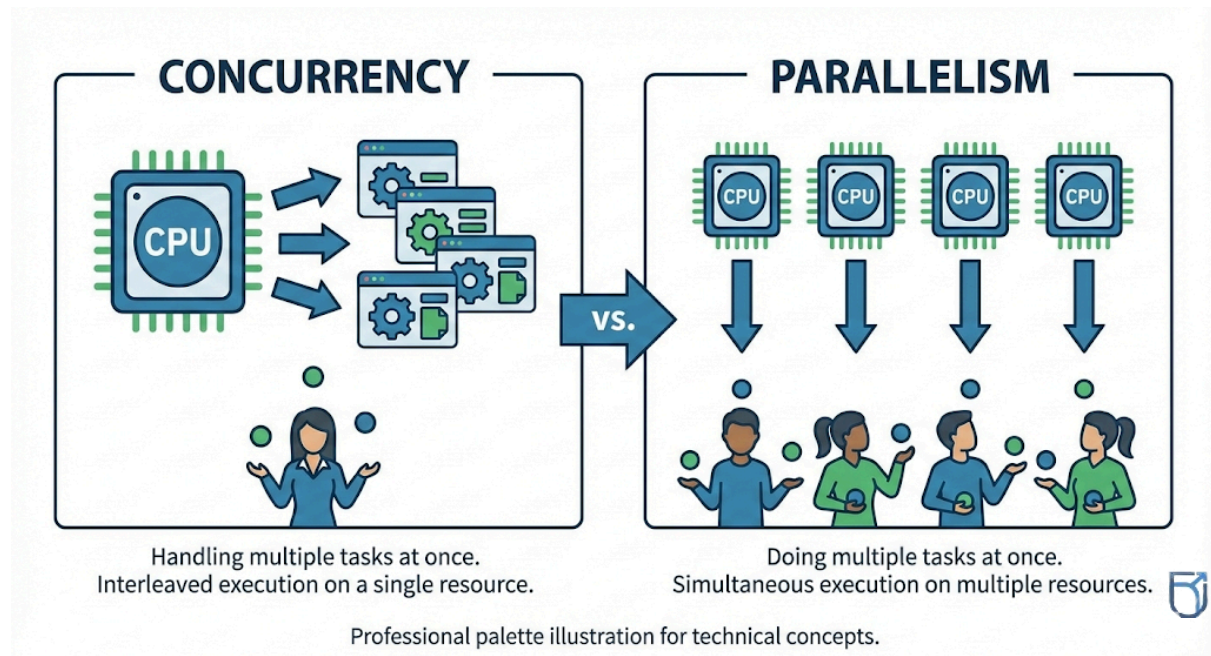
Computer operations are not magic; they happen in steps (Read → Modify → Write).

1. **Process A reads** the balance: **\$100**.
2. *Context Switch happens! The CPU pauses A and switches to B.*
3. **Process B reads** the balance: **\$100** (Because A hasn't saved the change yet!).
4. **Process B calculates** new balance: $\$100 - \$50 = \$50$.
5. **Process B writes** **\$50** to the database.
6. *Context Switch back to A.*
7. **Process A calculates** (using the old **\$100** it remembers): $\$100 - \$50 = \$50$.
8. **Process A writes** **\$50** to the database.

Result:

You withdrew \$50 twice (total \$100 taken), but the bank balance says \$50 remains instead of \$0. The bank has lost money!

Concurrency vs. Parallelism



Feature	Concurrency	Parallelism
Concept	Dealing with multiple things at once.	Doing multiple things at once.
CPU	Single Core (mostly).	Multi-Core.
How it works	The CPU switches between tasks so fast (milliseconds) it <i>looks</i> like they are running together.	Different CPUs run different tasks at the exact same time.
Analogy	One chef chopping onions, then stirring sauce, then checking the oven (interleaving).	Two chefs: one chops onions while the other stirs the sauce (simultaneous).

Critical Section Problem

Definition: The Critical Section (CS) is a specific segment of code in a process where the process accesses and modifies **shared resources** (like common variables, files, or database tables).

The Goal: The primary goal is **Mutual Exclusion**: If one process is executing inside its Critical Section, no other process is allowed to enter their Critical Sections for the same shared resource.

Structure of a Process: To handle synchronization, we wrap the critical code with safety mechanisms. A typical process is divided into four parts:

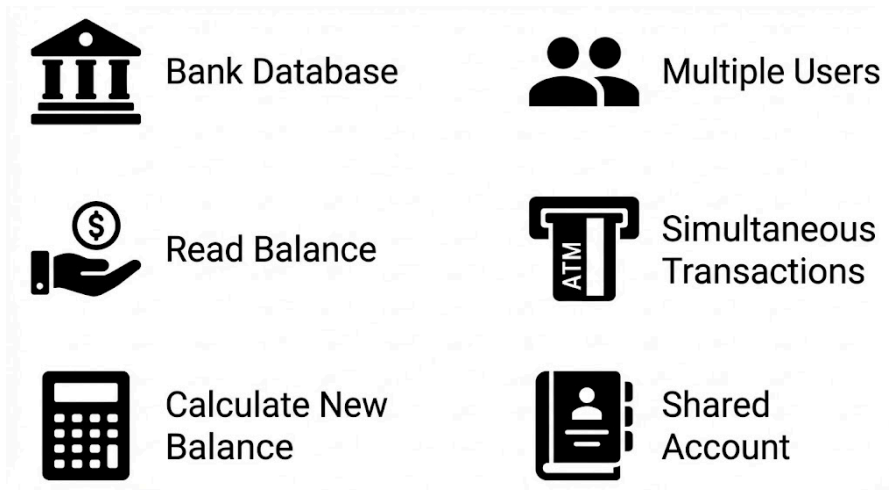
1. **Entry Section:** The code that requests permission to enter the critical section. It checks if it's safe to proceed (i.e., is the resource free?).
2. **Critical Section (CS):** The actual code where shared data is read or written. This part must be executed quickly.
3. **Exit Section:** The code that releases the permission (e.g., unlocking a lock), letting other processes know the resource is free.
4. **Remainder Section:** The rest of the code that does not involve shared resources.

CRITICAL SECTION - Multiple threads access this!

```
int balance = read_balance(); // 📖 Read
balance = balance - amount;   // 🧮 Calculate
write_balance(balance);       // ✍️ Write
```

Why is this critical?

- Multiple users → Same account
- Simultaneous transactions
- Shared variable: balance



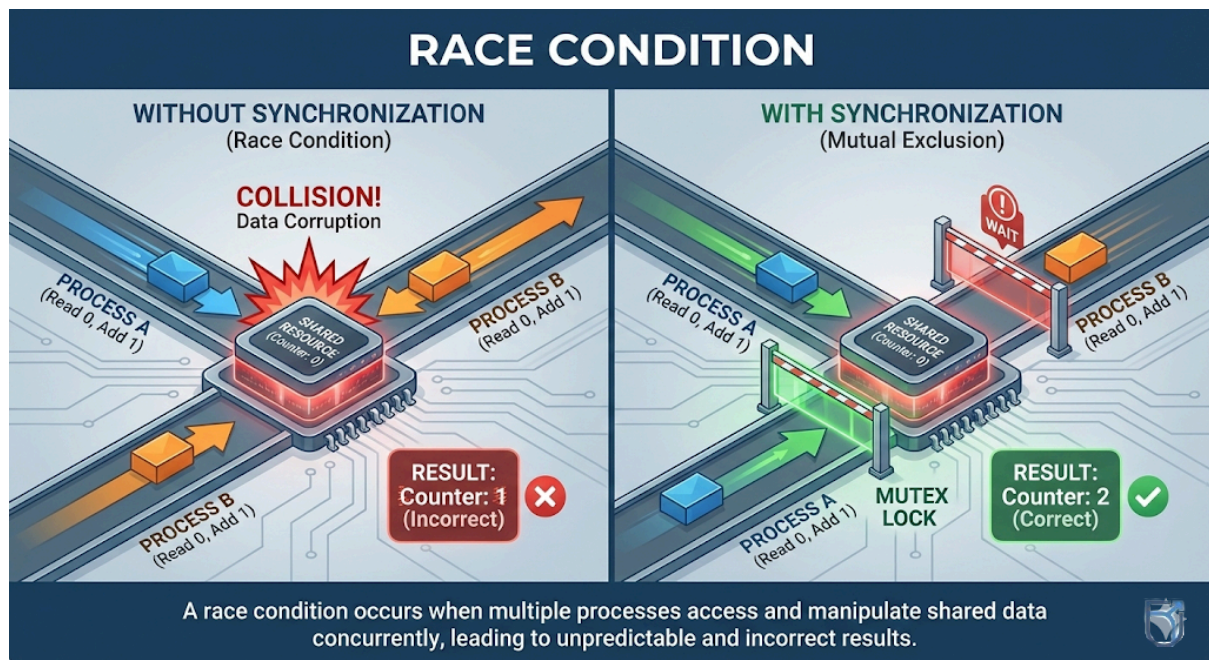
Critical Section Examples

These are common scenarios where synchronization is required because multiple processes try to perform a "Read-Modify-Write" cycle on the same data.

- **Bank Account Update:**
 - Two users accessing the same account via an ATM and an App simultaneously. If not synchronized, a withdrawal might be lost (as seen in the previous note).
- **Shared Counter Increment (`count++`):**
 - In high-level languages, `count++` looks like one step.
 - In assembly (CPU level), it is three steps:
 1. Load `count` from memory to register.
 2. Add 1 to register.
 3. Store register back to memory.
 - If a context switch happens between steps 1 and 3, the count will be wrong.
- **Producer-Consumer Shared Buffer:**
 - One process (Producer) adds data to a fixed-size buffer; another (Consumer) removes it. Without sync, the Producer might overwrite data the Consumer hasn't read yet, or the Consumer might read empty garbage data.
- **Linked List Insertion:**
 - If two threads try to add a node to the same linked list at the same time, the `next` pointers might get messed up, causing one of the new nodes to be detached and lost forever (memory leak).

Race Condition

Definition: A Race Condition occurs when the final outcome of a program depends on the **timing** or the **order of execution** of parallel threads. It is a "race" because the threads are competing to write to the memory last.



```
For Deposit (Process A):
int balance = read_balance(); // reads 5000
balance = balance + 1000;      // calculates 6000
write_balance(balance);        // writes 6000

For Withdrawal (Process B):
int balance = read_balance(); // reads 5000
balance = balance - 2000;      // calculates 3000
write_balance(balance);
    // writes 3000
Expected Final Balance: 4000 (5000 + 1000 - 2000)
```

Bounded Buffer Problem Setup:

Shared buffer with counter variable tracking number of items.

Producer Code:

```

while (true) {
    // produce item
    while (counter == BUFFER_SIZE)
        ; // buffer full, wait
    buffer[in] = item;
    in = (in + 1) % BUFFER_SIZE;
    counter++; // CRITICAL OPERATION
}

```

Consumer Code:

```

while (true) {
    while (counter == 0)
        ; // buffer empty, wait
    item = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--; // CRITICAL OPERATION
}

```

The Problem: *counter++ and counter-- are NOT atomic operations!*

Why counter++ is Dangerous

Looks Simple, Right?

counter++; // Just one line!

But In Machine Code Assembly

```

LOAD    R1, counter    // Step 1: Read
ADD     R1, R1, 1       // Step 2: Calculate
STORE   R1, counter    // Step 3: Write

```

3 separate instructions = 3 points where interruption can occur!

The Scenario:

Initial: counter = 5

Thread 1: counter++ (wants: 6)

Thread 2: counter++ (wants: 7)

✗ Race Condition Execution

T1: LOAD R1, counter	(R1 = 5)
T2: LOAD R2, counter	(R2 = 5)
T1: ADD R1, R1, 1	(R1 = 6)
T2: ADD R2, R2, 1	(R2 = 6)
T1: STORE R1, counter	(counter = 6)
T2: STORE R2, counter	(counter = 6)

Expected: 7

Actual: 6

One increment lost!

Key Characteristics:

- **Non-Deterministic:** You run the program once, and it works. You run it again, and it crashes or gives a different result. This makes it incredibly dangerous.
- **Difficult to Debug:** Because the error depends on precise timing (e.g., a context switch happening at the exact wrong nanosecond), the bug often disappears when you try to debug it (sometimes called a "Heisenbug").
- **Data Corruption:** The shared data ends up in an inconsistent state (like the bank balance being incorrect).

Race Condition Examples

A race condition happens when the system's behavior depends on the unfortunate timing of events. Here are the classic scenarios:

- **Bank Deposit + Withdrawal:** As discussed, if a deposit (\$100) and withdrawal (\$50) happen simultaneously without locking, the system might overwrite the deposit, and the balance ends up being just -\$50 instead of +\$50.
- **Counter Increment (counter++):** The "Hello World" of race conditions. Two threads trying to increase a visitor count from 10 to 11 might both read "10" and both write "11," effectively losing one visitor.
- **Producer-Consumer Counter:** The producer increases the "items in buffer" count, and the consumer decreases it. If they race, the count might become negative or exceed the buffer size.
- **Linked List Corruption:** If two threads try to insert a node at the same time, they might overwrite each other's pointers. The result is a broken list where some nodes are unreachable (memory leak) or the list points to garbage.
- **TOCTOU (Time-Of-Check to Time-Of-Use):** A security-specific race condition.
 - *Example:* A program checks if you have permission to edit a file (Time-Of-Check). milliseconds later, it opens the file (Time-Of-Use). In

that tiny gap, a hacker swaps the file for a protected system file. The program, thinking it's safe, overwrites the system file.

Why Operations Are Not Atomic

"Atomic" means "indivisible"—it happens all at once or not at all. High-level code (like C++ or Java) is **not** atomic.

The `counter++` Breakdown: To the CPU, `counter++` is actually three separate instructions:

1. **LOAD:** Move value from RAM to a CPU Register.
2. **MODIFY:** Add 1 to the value in the Register.
3. **STORE:** Move the new value from Register back to RAM.

The Lost Update Problem: If the Operating System switches context (interleaving) after step 1 but before step 3, the second process uses old data. When the first process comes back, it overwrites the work of the second process. The update is "lost."

Consequences of Race Conditions

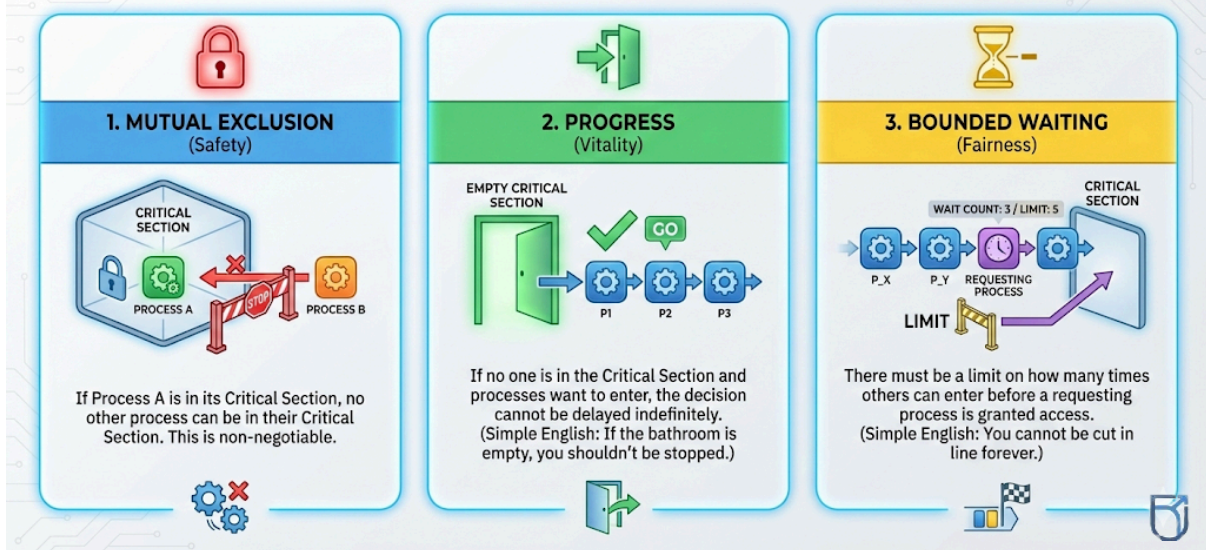
- **Data Corruption:** Database records become inaccurate.
- **Lost Updates:** As seen in the counter example.
- **Deadlocks:** In an attempt to fix race conditions incorrectly, processes may end up waiting for each other forever.
- **Starvation:** A process might get blocked indefinitely while others proceed.
- **System Crashes:** If a pointer in a kernel data structure is corrupted, the whole OS can panic/crash.
- **Security Vulnerabilities:** Hackers exploit race conditions (like TOCTOU) to gain root access.

Requirements for Critical Section Solutions (The MPB Rule)

For any solution (algorithm or hardware) to correctly solve the Critical Section problem, it **must** satisfy these three criteria:

REQUIREMENTS FOR CRITICAL SECTION SOLUTIONS (THE MPB RULE)

The three criteria any correct algorithm or hardware solution must satisfy.



1. Mutual Exclusion (Safety):

- If Process A is in its Critical Section, **no other process** can be in their Critical Section. This is non-negotiable.

2. Progress (Vitality):

- If no one is in the Critical Section and some processes *want* to enter, those processes cannot be stopped indefinitely. The decision of who goes next cannot be delayed forever.
- *Simple English*: If the bathroom is empty, you shouldn't be stopped from using it just because someone else *might* want to use it later.

3. Bounded Waiting (Fairness):

- There must be a limit (bound) on how many times other processes are allowed to enter their Critical Sections before a requesting process is granted access.
- *Simple English*: You cannot be cut in line forever. Eventually, you must get a turn.

Mutual Exclusion

- **Definition**: The guarantee that only **one** execution thread can enter the Critical Section at a time.
- **Role**: It is the primary guard against data corruption.
- **Implementation**: It is usually achieved using locks (Mutex) or Semaphores. If a process holds the lock, all others are blocked until the lock is released.

Progress

Definition: Progress ensures that if the Critical Section (CS) is empty, processes are not stopped from entering it indefinitely.

Key Rules:

- **No Deadlock:** If no one is using the shared resource, and a process wants to use it, it must be allowed to enter.
- **Decision Makers:** Only processes that are *actively trying* to enter the CS get to participate in the decision of who goes next.
- **No Blocking by Outsiders:** A process operating in its "Remainder Section" (doing unrelated work) cannot stop another process from entering the CS.

Simple Explanation: If the room is empty, you don't need permission from someone who isn't even in the building. You can go in.

Bounded Waiting

Definition: There must be a limit (a bound) on the number of times other processes are allowed to enter the Critical Section after a process has made a request to enter.

Key Rules:

- **Prevents Starvation:** Ensures that a process doesn't wait forever while others keep jumping the queue.
- **Guarantees Fairness:** Every process will eventually get its turn.

Simple Explanation: If you are waiting in line, others can't keep cutting in front of you indefinitely. Eventually, the line must move, and you must get served.

Additional Desirable Properties

Beyond the mandatory MPB rules (Mutual Exclusion, Progress, Bounded Waiting), a good synchronization solution should have:

- **Performance:** The mechanism (like a lock) should be fast and not waste CPU cycles.
- **Fairness:** No single process should dominate the resource (related to Bounded Waiting).
- **Simplicity:** The algorithm should be easy to understand and implement to avoid bugs.
- **Portability:** It should work correctly across different operating systems and hardware architectures.

Failed Synchronization Attempts

Before modern solutions were invented, early software attempts often failed because they couldn't satisfy all three requirements (Mutual Exclusion, Progress, Bounded Waiting).

1. Strict Alternation (Turn Variable):

- **Method:** Uses a **turn** variable (0 or 1). Processes strictly take turns ($A \rightarrow B \rightarrow A \rightarrow B$).
- **Failure:** Violates **Progress**. If Process A is blocked or doing other work, Process B cannot enter, even if the resource is free.

2. Flag-Based Solution:

- **Method:** Each process raises a "flag" to signal it wants to enter.
- **Failure:** Causes **Deadlock**. If both raise flags at the exact same time, they both wait for the other to lower the flag. Neither enters.

Concurrency in Modern Systems

Modern hardware makes synchronization harder than it used to be on older, single-CPU systems.

Challenges:

- **Multi-Core Processors:** True parallelism means two cores can modify memory at the exact same instant. Software tricks aren't enough; we need hardware instructions.
- **Cache Coherence:** Each CPU core has its own cache. If Core 1 updates a value in its cache but doesn't write it to main RAM immediately, Core 2 will see old data.
- **Memory Ordering:** Modern CPUs reorder instructions to run faster. This optimization can break synchronization logic (e.g., executing the "unlock" instruction before the "write" instruction is finished).

OS Areas Using Synchronization

Operating Systems use synchronization internally to keep the system stable. It's not just for user programs; the OS itself needs protection.

- **Process Management:**
 - The OS maintains a list of all running processes (Process Control Block - PCB). Synchronization ensures two processes don't try to update the status of the same PCB at the same time.



- **Memory Management:**
 - When multiple processes ask for RAM, the OS must synchronize the "free memory" list to avoid giving the same block of RAM to two different programs.
- **File Systems:**
 - Prevents data corruption when multiple users or programs try to write to the same file or directory simultaneously.
- **Device Management:**
 - Manages access to hardware like printers. It ensures that print jobs from different users are queued one by one, rather than printing lines from two documents on the same page.
- **Scheduler Queues:**
 - The Ready Queue (where processes wait for CPU) is a shared resource. The OS uses locking to safely add or remove processes from this queue.

Classic Synchronization Problems

These are standard theoretical problems used to test if a synchronization algorithm works correctly.

1. Producer–Consumer Problem (Bounded Buffer)

- **Scenario:** One process (Producer) generates data and puts it into a fixed-size buffer. Another process (Consumer) takes data out.
- **The Challenge:** The Producer must not add data if the buffer is full, and the Consumer must not remove data if the buffer is empty.
- **Goal:** Correctly managing the count of items in the buffer.

2. Readers–Writers Problem

- **Scenario:** A database or file is shared. "Readers" only look at data; "Writers" modify it.
- **The Challenge:**
 - Multiple Readers can read at the same time (safe).
 - If a Writer is writing, *no one else* (neither Reader nor Writer) can access the data.
- **Goal:** Preventing writers from starving readers, or vice-versa.

3. Dining Philosophers Problem

- **Scenario:** 5 philosophers sit at a round table with 5 forks. To eat, a philosopher needs *two* forks (left and right).
- **The Challenge:** If everyone picks up their left fork at the same time, no one can pick up their right fork. Everyone waits forever.
- **Goal:** Preventing **Deadlock** (stuck forever) and **Starvation** (never getting to eat).

4. Sleeping Barber Problem

- **Scenario:** A barbershop has one barber, one barber chair, and a waiting room with limited seats.
- **The Challenge:**
 - If no customers are there, the barber sleeps.
 - If a customer arrives and the barber is sleeping, they wake him up.
 - If the barber is busy, the customer waits in a chair.
 - If no chairs are free, the customer leaves.
- **Goal:** Coordinating the sleeping/waking cycle and queue management without race conditions.

Preview of Synchronization Techniques

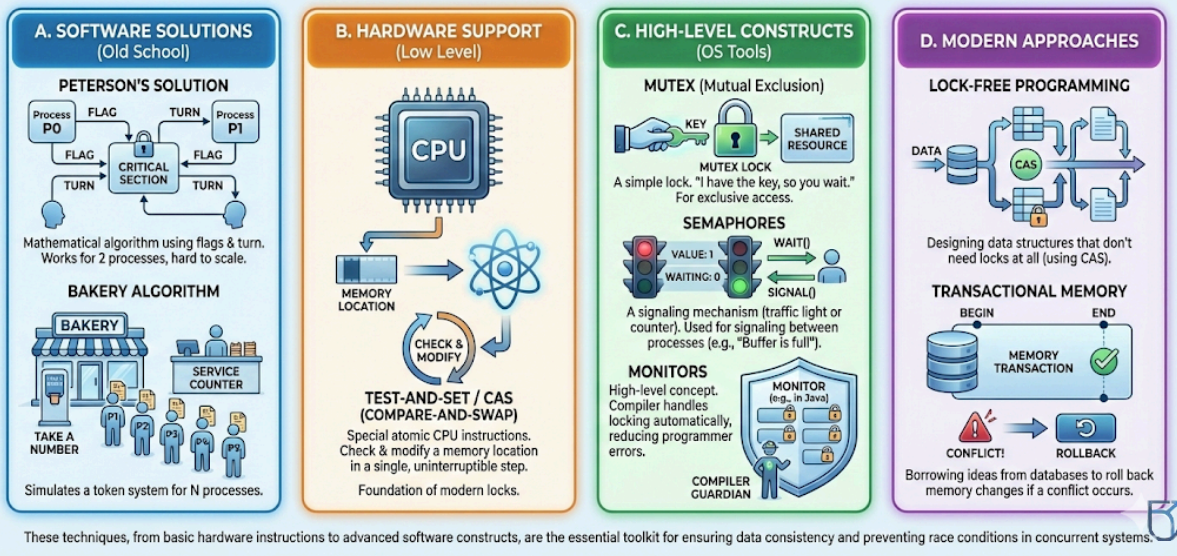
This is an overview of the tools used to solve the problems above.

A. Software Solutions (Old School):

- **Peterson's Solution:** A mathematical algorithm using flags and turn variables. Works well for 2 processes but is hard to scale.
- **Bakery Algorithm:** Simulates a token system (like a bakery) for N processes.

PREVIEW OF SYNCHRONIZATION TECHNIQUES

OVERVIEW OF TOOLS TO SOLVE CRITICAL SECTION PROBLEMS & PREVENT RACE CONDITIONS



B. Hardware Support (Low Level):

- **Test-and-Set / Compare-And-Swap (CAS):** Special atomic CPU instructions. They check and modify a memory location in a single, uninterruptible step. This is the foundation of modern locks.

C. High-Level Constructs (OS Tools):

- **Mutex (Mutual Exclusion):** A simple lock. "I have the key, so you wait until I give it back." Used for exclusive access.
- **Semaphores:** A signaling mechanism (like a traffic light or a counter). Used for signaling between processes (e.g., "Buffer is full").
- **Monitors:** A high-level programming concept (used in Java) where the compiler handles the locking automatically, making it harder for programmers to make mistakes.

D. Modern Approaches:

- **Lock-Free Programming:** designing data structures that don't need locks at all (using CAS).
- **Transactional Memory:** borrowing ideas from databases to roll back memory changes if a conflict occurs.